

# Laboratório 05

## Partes 4 e 5

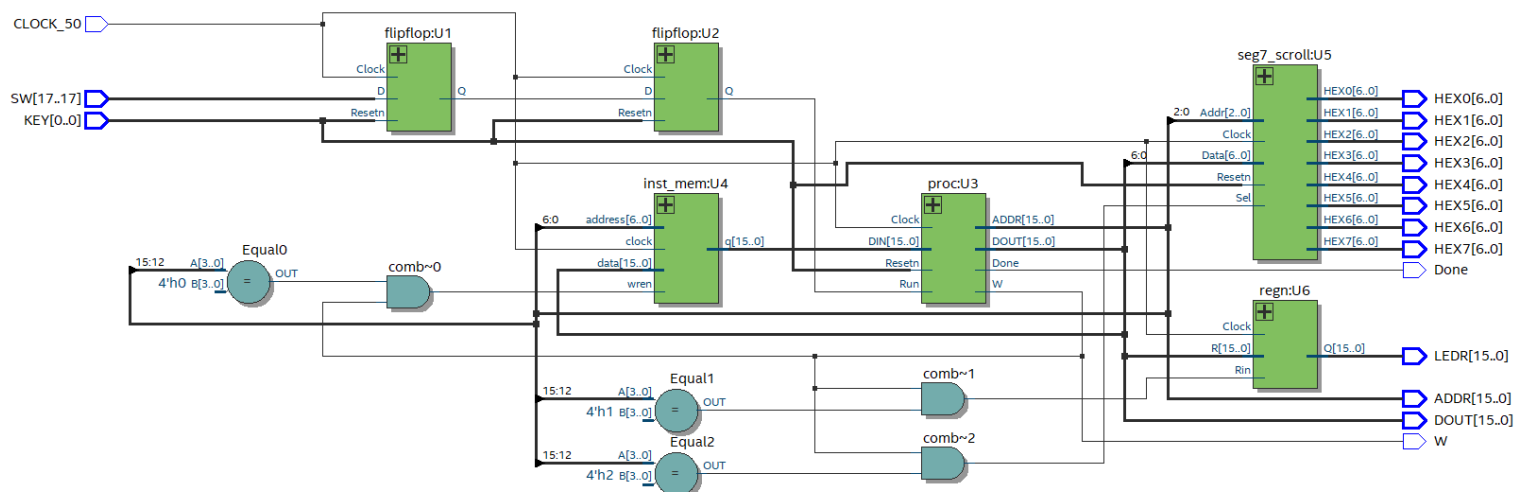
Neste laboratório o objetivo é implementar um microcontrolador simples de 16 bits. O laboratório está dividido em 5 partes, e neste documento iremos detalhar as duas últimas destas.

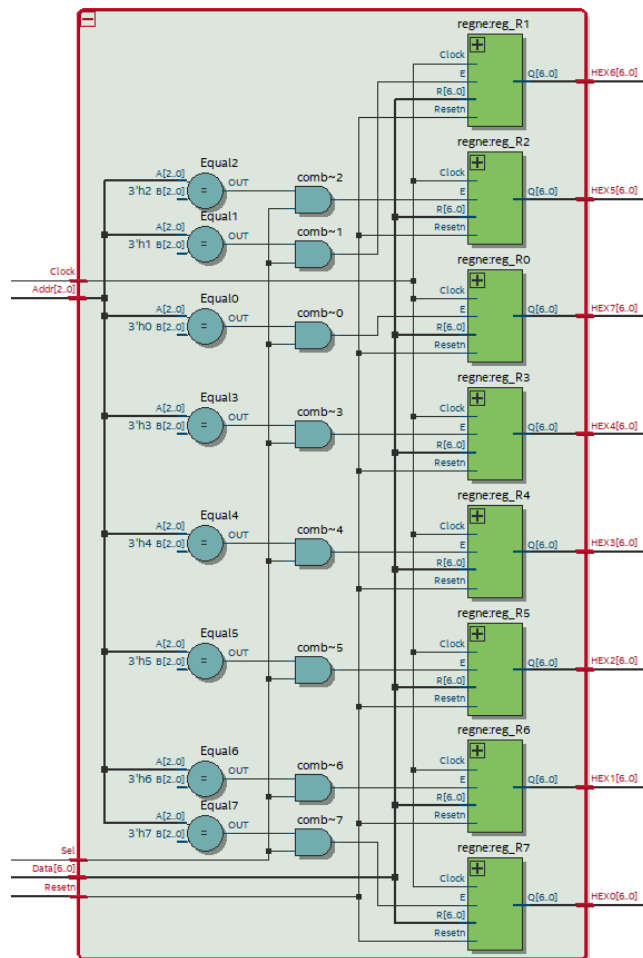
### Parte 4

Nesta parte você deve implementar uma forma de se permitir efetuar a operação de **st** nos displays de 7 segmentos, de forma análoga ao que foi feito com os LEDs na parte 3. A construção do circuito será feita de forma semelhante. Ou seja, o conjunto de displays será endereçado por uma faixa de endereços, de tal forma que  $A_{15}A_{14}A_{13}A_{12} = 0010$ , e, nesta condição, cada display individualmente será mapeado por seu número em  $A_2A_1A_0$ . Ou seja, a faixa de endereçamento dos displays de 7-segmentos é da forma 0010 XXXX XXXX XABC, onde X são estados de Don't Care e:

- ABC = "000" endereça o HEX0;
- ABC = "001" endereça o HEX1;
- ...;
- ABC = "111" endereça o HEX7.

Cada display de 7-segmentos deve assim ter sua entrada alimentada por um registrador, que, por sua vez, armazena os 7 bits menos significativos dos dados durante a operação de **st** endereçado a ele. Assim, a conexão final se aproxima do que é apresentado nas próximas duas figuras. Na primeira temos uma visão total do circuito, onde, nos atentando ao bloco seg7\_scroll, podemos observar a faixa de entrada dos dados (obtidas a partir dos 7 bits menos significativos de DOUT), a faixa de endereço (3 bits menos significativos de ADDR), clock e resetn compartilhados, e um sinal de Enable que está ativo quando W está ativo e os 4 bits mais significativos de ADDR são iguais a 4'h2 (ou seja, 2 em hexadecimal com 4 bits -> 0010). Na segunda figura, mostra-se um zoom deste componente, destacando os registradores.



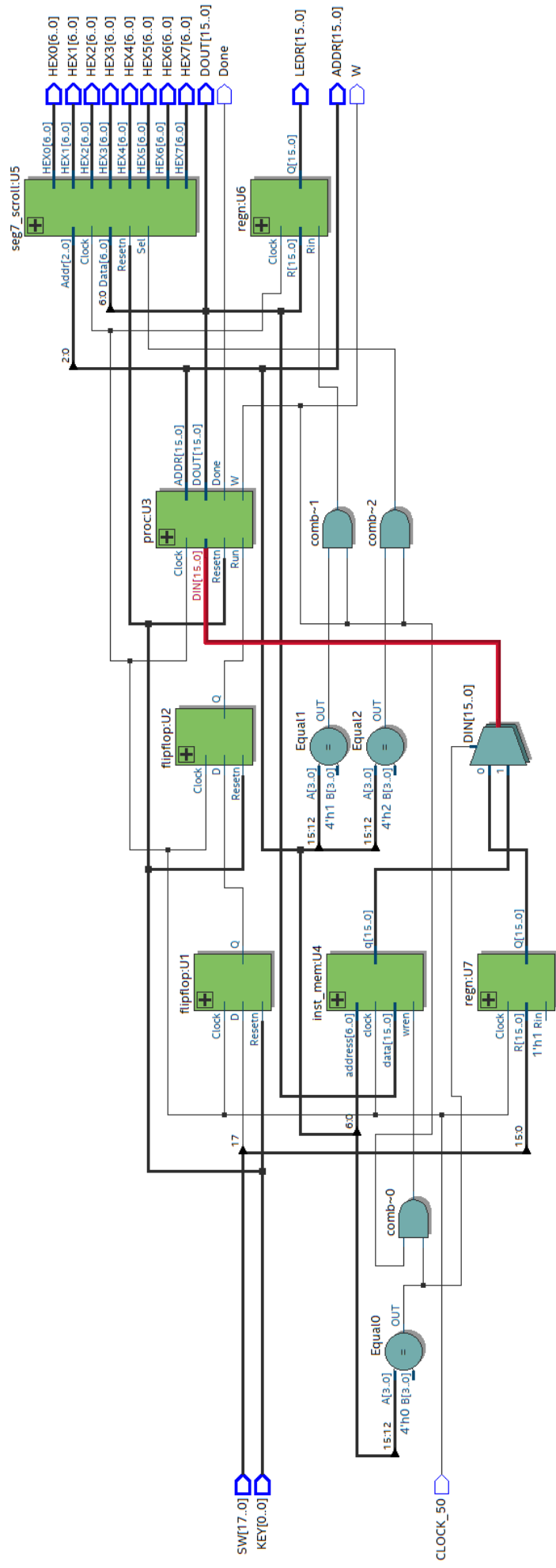


Não serão feitas simulações temporais para esta parte, pois iremos testar esta conexão no código final do processador. De toda forma, apresente no relatório imagens semelhantes a estas duas últimas, que podem ser produzidas no Quartus em *Tools->Netlist Viewers->RTL Viewer*. Justifique que seu circuito implementa a funcionalidade correta.

## Parte 5

Nesta parte você deve implementar uma forma de se permitir efetuar a operação de **ld** no valor representado pela posição das chaves ( $SW_{15}-SW_0$ ). Este conjunto de dados será endereçado pela faixa  $A_{15}A_{14}A_{13}A_{12} = 0011$ . Ou seja, a faixa de endereçamento do conjunto de chaves é da forma  $0011\ XXXX\ XXXX\ XXXX$ , onde X são estados de Don't Care.

Observe que DIN pode ser alimentado por 2 fontes: a saída da memória, quando  $A_{15}A_{14}A_{13}A_{12} = 0000$ , e os valores das chaves, quando  $A_{15}A_{14}A_{13}A_{12} = 0011$ . Desta forma, deve-se implementar um multiplexador que alimenta DIN a partir da saída da memória e do valor das chaves, cuja seleção se dá por estes endereços.



Idealmente, como apresentado na Figura anterior, os valores das Chaves devem ser alimentados ao multiplexador através da saída de um registrador, de forma que todos os sinais do circuito permaneçam sincronizados.

A entidade final utilizada é

```
ENTITY lab12_part5 IS
    PORT ( KEY          : IN      STD_LOGIC_VECTOR(0 DOWNT0 0);
          SW            : IN      STD_LOGIC_VECTOR(17 DOWNT0 0);
          CLOCK_50      : IN      STD_LOGIC;
          HEX7, HEX6, HEX5,
          HEX4, HEX3, HEX2,
          HEX1, HEX0    : OUT     STD_LOGIC_VECTOR(6 DOWNT0 0);
          LEDR          : OUT     STD_LOGIC_VECTOR(15 DOWNT0 0);
          DOUT, ADDR    : BUFFER STD_LOGIC_VECTOR(15 DOWNT0 0);
          W, Done       : BUFFER STD_LOGIC);
END lab12_part5;
```

onde Run é conectado em SW<sub>17</sub> e Reseth em KEY<sub>0</sub>. O Arquivo .mif utilizado é apresentado a seguir:

```
DEPTH = 128;
WIDTH = 16;
ADDRESS_RADIX = HEX;
DATA_RADIX = BIN;
CONTENT BEGIN

00 : 0010000000000000;          % mvi    R0,#1                K %
01 : 0000000000000001;
02 : 0000010000000000;          % mv     R1,R0                1 %
03 : 0011100000000000;          % mvi    R6,#Beta Q <- '*'D' %
04 : 0000000001100110;
05 : 0010100000000000;          % mvi    R2,#0                [LED] %
06 : 0000000000000000;
07 : 0001011100000000;          % Loop   mv     R5,R6                P <- Q %
08 : 0011000000000000;          % mvi    R4,#H7_address %
09 : 0010000000000000;
0A : 1000111010000000;          % ld     R3,R5                [P] %
0B : 1010111000000000;          % st     R3,R4                H7 <- [P] %
0C : 0101010010000000;          % add    R5,R1                P++ %
0D : 0101000010000000;          % add    R4,R1                H6 %
0E : 1000111010000000;          % ld     R3,R5                [P] %
0F : 1010111000000000;          % st     R3,R4                H6 <- [P] %
10 : 0101010010000000;          % add    R5,R1                P++ %
11 : 0101000010000000;          % add    R4,R1                H5 %
12 : 1000111010000000;          % ld     R3,R5                [P] %
13 : 1010111000000000;          % st     R3,R4                H5 <- [P] %
14 : 0101010010000000;          % add    R5,R1                P++ %
15 : 0101000010000000;          % add    R4,R1                H4 %
16 : 1000111010000000;          % ld     R3,R5                [P] %
17 : 1010111000000000;          % st     R3,R4                H4 <- [P] %
18 : 0101010010000000;          % add    R5,R1                P++ %
19 : 0101000010000000;          % add    R4,R1                H3 %
1A : 1000111010000000;          % ld     R3,R5                [P] %
1B : 1010111000000000;          % st     R3,R4                H3 <- [P] %
1C : 0101010010000000;          % add    R5,R1                P++ %
1D : 0101000010000000;          % add    R4,R1                H2 %
1E : 1000111010000000;          % ld     R3,R5                [P] %
1F : 1010111000000000;          % st     R3,R4                H2 <- [P] %
```

```

20 : 0101010010000000;
21 : 0101000010000000;
22 : 1000111010000000;
23 : 1010111000000000;
24 : 0101010010000000;
25 : 0101000010000000;
26 : 1000111010000000;
27 : 1010111000000000;
28 : 0111100000000000;
29 : 0011010000000000;
2A : 0000000001100000;
2B : 0011000000000000;
2C : 0000000000110110;
2D : 0111011100000000;
2E : 1101111000000000;
2F : 0101100010000000;
30 : 0101100010000000;
31 : 0011010000000000;
32 : 1111111111111111;
33 : 0111010000000000;
34 : 0101010010000000;
35 : 0000001010000000;
36 : 0011010000000000;
37 : 0000000001100111;
38 : 0011000000000000;
39 : 0000000001000011;
3A : 0111011100000000;
3B : 1101111000000000;
3C : 0111100010000000;
3D : 0111100010000000;
3E : 0011010000000000;
3F : 1111111111111111;
40 : 0111010000000000;
41 : 0101010010000000;
42 : 0000001010000000;
43 : 0011010000000000;
44 : 0000000001110000;
45 : 1010001010000000;
46 : 0010110000000000;
47 : 0001000000000000;
48 : 1010100110000000;
49 : 0100100010000000;
4A : 0010110000000000;
4B : 0000000000000001;
4C : 0001011110000000;
4D : 0010000000000000;
4E : 0011000000000000;
4F : 1001000000000000;
50 : 0000001110000000;
51 : 0111000010000000;
52 : 1101110000000000;
53 : 0110110010000000;
54 : 1101111010000000;
55 : 0011010000000000;
56 : 0000000001110000;
57 : 1000001010000000;
58 : 0011110000000000;
59 : 0000000000000111;
60 : 0000000000000000;
61 : 1111111111111111;
62 : 1111111111111111;
63 : 1111111111111111;
64 : 1111111111111111;
65 : 1111111111111111;
66 : 0000000001000010;
67 : 0000000000110000;
68 : 0000000000010010;
69 : 1111111111111111;

% add    R5,R1          P++ %
% add    R4,R1          H1 %
% ld     R3,R5          [P] %
% st     R3,R4          H1 <- [P] %
% add    R5,R1          P++ %
% add    R4,R1          H0 %
% ld     R3,R5          [P] %
% st     R3,R4          H0 <- [P] %
% sub    R6,R0          Q <- Q - K %
% mvi    R5,#Alpha-1 %

% mvi    R4,#Skip %

% sub    R5,R6          Q == Alpha-1? %
% mvnz   R7,R4          No %
% add    R6,R1 %
% add    R6,R1          Q <- Alpha+1 %
% mvi    R5,#1111111111111111 %

% sub    R5,R0 %
% add    R5,R1 %
% mv     R0,R5          K <- -K %
% Skip   mvi    R5,#Beta+1 %

% mvi    R4,#Cont %

% sub    R5,R6          Q == Beta+1? %
% mvnz   R7,R4          No %
% sub    R6,R1 %
% sub    R6,R1          Q <- Beta-1 %
% mvi    R5,#1111111111111111 %

% sub    R5,R0 %
% add    R5,R1 %
% mv     R0,R5          K <- -K %
% Cont   mvi    R5,#Temp Save reg %

% st     R0,R5 %
% mvi    R3,#LED LED reg address %

% st     R2,R3          [LED] %
% add    R2,R1          ++[LED] %
% mvi    R3,#000000000000001 Delay %

% mv R5,R7 Save addr. of next instruction %
% Outer mvi R0,#SW Read switch valuation %

% ld     R4,R0          Nested delay %
% mv R0,R7 Save addr. of next instruction %
% Inner  sub R4,R1      Decrement R4 %
% mvnz   R7,R0          jnz Inner %
% sub    R3,R1          Decrement R3 %
% mvnz   R7,R5          jnz Outer %
% mvi    R5,#Temp      Restore regs %

% ld     R0,R5 %
% mvi    R7,#Loop %

% Alpha-1
% Alpha      ' ' %
%            ' ' %
%            ' ' %
%            ' ' %
%            ' ' %
% Beta       'd' %
% Beta+1     'E' %
%            '2' %
%            ' ' %

```

END;

Observe o efeito de *zig-zag* nas informações do display, e a temporização sendo variável a partir da mudança dos valores de  $SW_{15}$ - $SW_0$ . É possível observar a mudança de temporização nos LEDs também.